STA426: Statistical Analysis of High-Throughput Genomic and Transcriptomic Data

Semester project: Clustering analysis

Joël Meili, Felix Hofmann

January 10, 2020

Table of contents

1 Introduction **2**

2 Methods **2**

2.1 Seurat 2

2.2 SC3 2

3 Results **3**

3.1 Simulation of the data sets 3

3.2 Exploratory data analysis 4

3.3 Clustering - Standard clustering techniques 5

3.4 Clustering - Seurat pipeline 11

3.5 Clustering - SC3 pipeline 17

4 Conclusion **24**

Session information **25**

References **27**

1 Introduction

Every cell of the human body contains the full genetic information, thus for the body to work the genes are expressed differently in the various cells. Standard RNA sequencing usually analyzes populations of cells or tissue samples, where differences might be mistaken for techincal nusiance such as library effects or batch effects, when in reality the discrepancy could be pinpointed to cell variety. Single cell RNA sequencing thus allows to investigate the level of gene expression from cells of the same tissue and differentiate which cells are expressed differently than others. In this short semester project the performance of known clustering techniques on scRNAseq data sets was investigated, and whether the real subpopulation of cells can be estimated by the use of said methods.

2 Methods

Two scRNAseq data sets consisting of four cell groups were simulated using the splatter package, implemented in the Bioconductor R environment. To evaluate the performance of the clustering techniques the data sets were manipulated to be easy respectively hard to cluster by adjusting the differentially expressed probability in the simulation process. For the cluster evaluation k-means and hierarchical clustering were used to assess the benchmark clustering performance, which was then compared to graph-based clustering using the Seurat pipeline and consensus-clustering implemented in the SC3-package. The assessment of overall clustering performance was evaluated by calculating the adjusted rand index (ARI) between the estimated clusters and the true subpopulations.

In the following subsections the methods used by Seurat and SC3 will briefly be explained since the k-means algorithm and hierarchic clustering are quite well known.

2.1 Seurat

The Seurat-package offers various methods for many different application but since the relevant part of the package for this project are the clustering functions, only these will be explained here. The usual workflow with respect to clustering starts with some standard pre-processing of the data which includes selection and filtration of cells based on quality control metrics, normalization, scaling and the detection of highly variable features. Normalization is usually done by log-normalizing the datasets using a size factor of 10'000 molecules for each cell per default. Next, the expression values for each gene across all cells are standardized through z-score transformation. Based on these standardized expression values, the variance of each gene across all cells is computed and per default the 2000 most variable genes are selected for further analysis. In a next step, PCA is applied to the features selected before and by default the first 10 PCs are selected for further analysis. Next, a K-nearest neighbor (KNN) graph is constructed in the PCA space based on the euclidean distance distance between the cells and edge weights are assigned based on Jaccard similarity. To cluster the cells, modularity optimization techniques such as the Louvain algorithm are applied to iteratively group cells together, with the goal of optimizing the standard modularity function. This procedure is implemented in the FindClusters function which needs a user specified resolution parameter as input. The choice of this parameter implicitly defines then how many clusters will result from the analysis. The Seurat package also offers tools to find marker genes for each cluster based on a variety of test measures like for example the ROC test. Furthermore, the package also provides lots of function to plot clustering results, differential expression or gene activity.

2.2 SC3

The methodology of used for the SC3 package consists of five steps:

In a first step, the data is filtered and genes that are either expressed in a low percentage of the cells in the dataset or a very high percentage of the cells. The reason for this is that ubiquitous and rare genes are most

often not very informative for the clustering. Furthermore, the gene filtering reduces the dimensionality of the data and thus speeds up the pipeline. The second step consists of the calculation of the distance matrices. These are calculated based on euclidean distance, pearson correlation and spearman correlation. In the third step, all the distance matrices are transformed using principal component analysis (PCA) or by calculating the eigenvectors of the associated graph Laplacian. The fourth step consists of a k-means clustering on the eigenvectors of the transformed distance matrices. In step 5, SC3 computes a consesus matrix using the Cluster-based Similarity Partitioning Algorithm. This algorithm constructs a binary similarity matrix for every clustering result where the actual cell labels are compared with the clustering result. Then, all these similarity matrices are averaged and result in a consensus matrix. This matrix is then clustered using hierarchical clustering with complete agglomeration and the clusters are inferred at the k level of hierarchy, where k is defined by the user.

3 Results

3.1 Simulation of the data sets

As a first step, the scRNAseq data sets were simulated with different sampling parameters to achieve one data set with good and one with bad separability. Each data set consists of 1000 cells and 10'000 features comprised by four cell groups. As a second step, the data sets were processed to not contain lowly expressed genes and the ERCC spike-ins. Further, the normalized and log-transformed counts were used to obtain the dimensional reduction matrices for both the PCA and uniform manifold approximation (UMAP).

```r
# load libraries
library(splatter)
library(scater)

# set seed for reproducibility
seed <- 123

# simulate single cell experiment using the splatter package
simulate_sce <- function(nGenes, batchCells, seed, group_prob, de_prob) {
  params <- newSplatParams()
  params <- setParams(params,  nGenes = nGenes,
                      batchCells = batchCells, seed = seed,
                      group.prob = group_prob,
                      de.prob = de_prob)

  sce <- splatSimulate(params, method = "groups", verbose = FALSE)

  # remove undetected genes
  sce <- sce[rowSums(counts(sce) > 0) > 0, ]

  # calculate per-cell quality control metrics
  qc <- perCellQCMetrics(sce)

  # remove cells with few or many detected genes
  ol <- isOutlier(qc$detected, nmads = 2, log = TRUE)
  sce <- sce[, !ol]

  # remove lowly expressed genes
  sce <- sce[rowSums(counts(sce) > 1) >= 10, ]
```

```r
  # remove ERCC spike-ins
  is.spike <- grepl("^ERCC", rownames(sce))
  sce <- sce[!is.spike, ]

  # compute sum-factors and normalize counts
  sce <- computeLibraryFactors(sce)
  sce <- logNormCounts(sce)

  # compute dimensional reduction
  sce <- runPCA(sce, exprs_values = "logcounts")
  sce <- runUMAP(sce, exprs_values = "logcounts")
  sce
}

# set parameters for simulation
nGenes <- 1e4
batchCells <- 1e3
group_simple = c(0.5, 0.1, 0.1, 0.3)
group_hard <- c(0.4, 0.1, 0.2, 0.3)
de_simple <- rep(0.08, 4)
de_hard <- c(0.02, 0.01, 0.02, 0.04)

# simulate data
sce_simple <- simulate_sce(nGenes = nGenes, batchCells = batchCells,
                           seed = seed, group_prob = group_simple,
                           de_prob = de_simple)
sce_hard <- simulate_sce(nGenes = nGenes, batchCells = batchCells,
                           seed = seed, group_prob = group_hard,
                           de_prob = de_hard)
```

3.2 Exploratory data analysis

The following dimensional reduction plots illustrate the differences in the two data sets. As mentioned earlier the goal of the data simulation was to obtain two datasets with varying degrees of separability. Both the PCA and UMAP plot of the first data set look very promising considering the possbility to achieve good clustering, whereas in the second dataset it looks more difficult to detect obvious clustering, especially in the UMAP plot (see Figure 2). Furthermore, Figure 1 shows the cell distribution by group and especially in the hard dataset there are relatively few cells in Group 2.

```r
# load libraries
library(ggplot2)
library(gridExtra)

# set global ggtheme
theme_set(theme_classic())
```

```r
# visualize cell distribution
grid.arrange(ggplot(data.frame(Group = sce_simple$Group), aes(x = Group)) +
               geom_bar() + ylim(0, 500) + ylab("Counts") +
               ggtitle("Cell distribution by group (simple)"),
             ggplot(data.frame(Group = sce_hard$Group), aes(x = Group)) +
               geom_bar() + ylim(0, 500) + ylab("Counts") +
```

```
              ggtitle("Cell distribution by group (hard)"),
            ncol = 2)
```

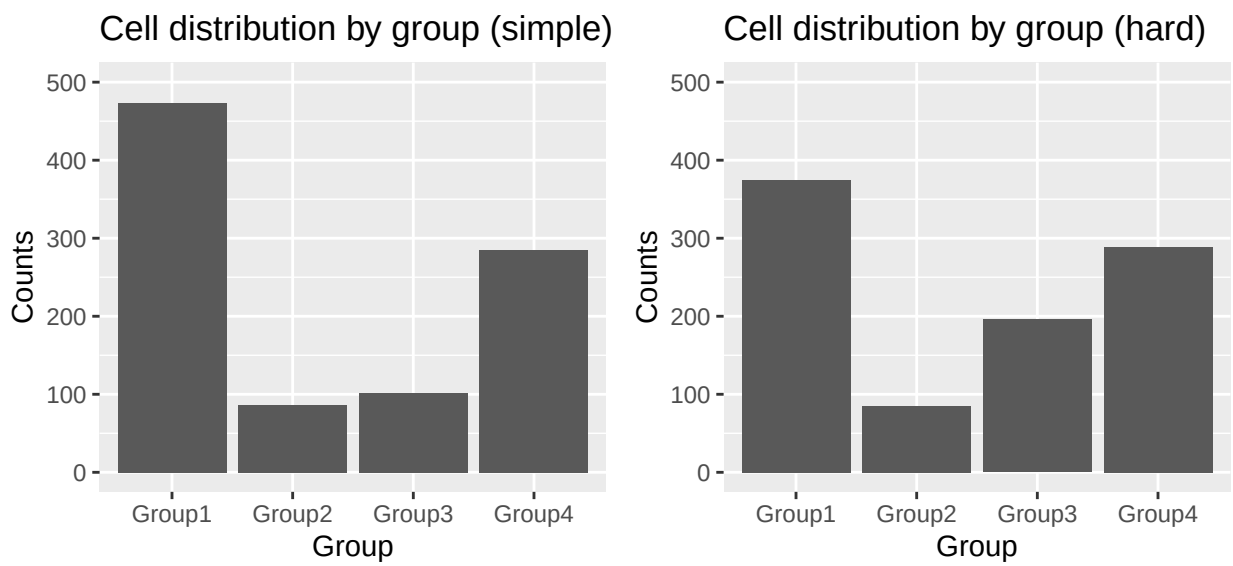

Figure 1: Cell distribution by group in simple and hard dataset.

```
# visualize dimensional reduction
grid.arrange(plotPCA(sce_simple, colour_by = "Group") +
               ggtitle("PCA visualization (simple)"),
             plotUMAP(sce_simple, colour_by = "Group") +
               ggtitle("UMAP visualization (simple)"),
             plotPCA(sce_hard, colour_by = "Group") +
               ggtitle("PCA visualization (hard)"),
             plotUMAP(sce_hard, colour_by = "Group") +
               ggtitle("UMAP visualization (hard)"),
             nrow = 2, ncol = 2)
```

3.3 Clustering - Standard clustering techniques

For the clustering by the above mentioned techniques the log-counts were transformed by principal component analysis. Then, k-means clustering was applied for every possible number of clusters between one and twenty for both data sets. The plot in Figure 3 shows that the correct number of clusters was detected for the first data set, but not for the second data set. Nevertheless, reasonable adjusted rand index scores were achieved for the second data set, although the cluster matching was not perfect. In Figure 4 one can see the agreement between the result of the k-means clustering and the true group membership.

```
# load libraries
library(parallel)
library(doParallel)
library(foreach)
library(clue)
library(mclust)
```

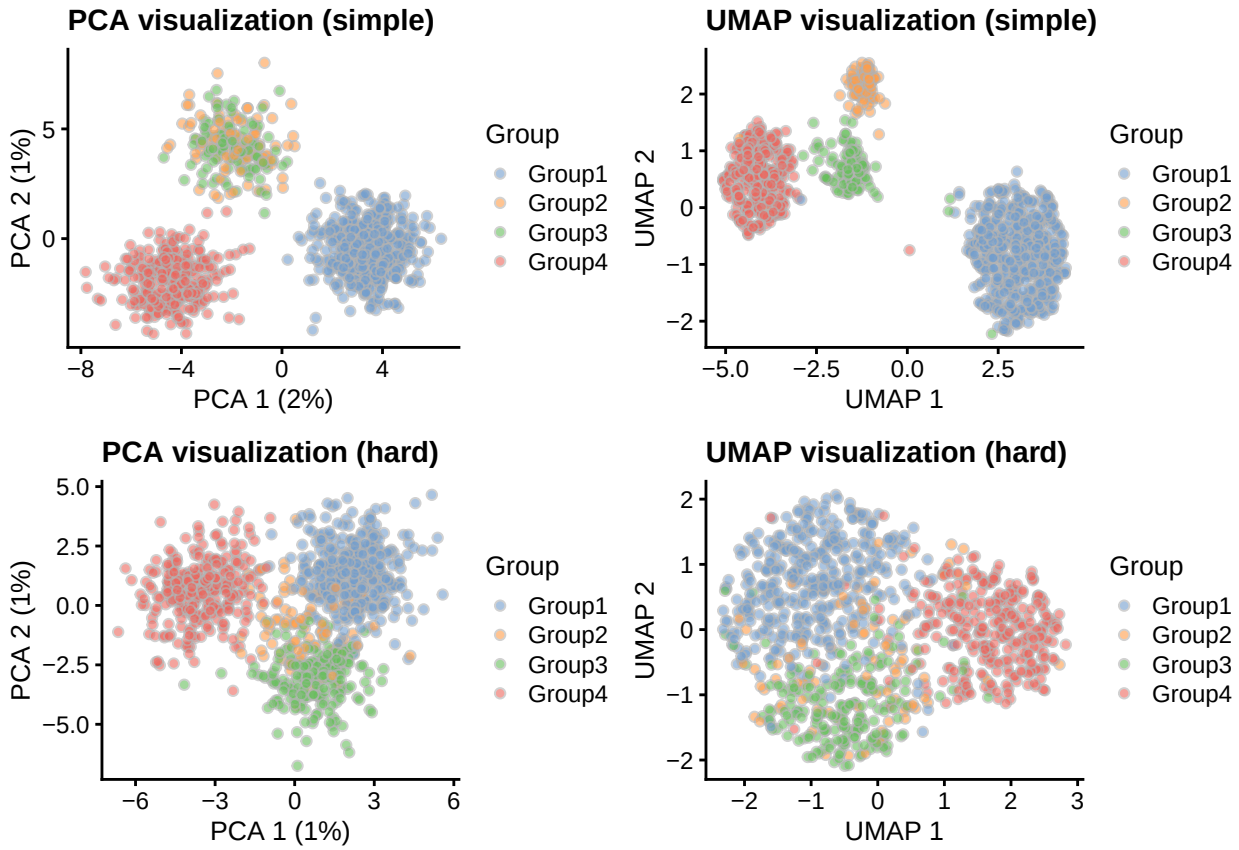

Figure 2: Different dimensional reduction techniques.

```r
# calculate pca of log-counts for clustering
pca_simple <- prcomp(t(logcounts(sce_simple)), center = TRUE, scale. = FALSE)
pca_hard <- prcomp(t(logcounts(sce_hard)), center = TRUE, scale. = FALSE)

# calculate euclidean distances for hierarchical clustering
dist_simple <- dist(pca_simple$x, method = "euclidean")
dist_hard <- dist(pca_hard$x, method = "euclidean")

# register parallel backend
registerDoParallel(detectCores() - 1)

# kmeans clustering
kmeans_clustering <- function(sce, pca) {
  adj_rand <- foreach(k = 1:20, .combine = "c") %dopar% {
    set.seed(seed)
    kout <- kmeans(pca$x, centers = k)
    adjustedRandIndex(sce$Group, kout$cluster)
  }
  adj_rand
}

kmeans_simple <- kmeans_clustering(sce_simple, pca_simple)
kmeans_hard <- kmeans_clustering(sce_hard, pca_hard)
```

```r
grid.arrange(ggplot(data.frame(k = 1:20, ARI = kmeans_simple), aes(x = k, y = ARI)) +
               geom_line() + geom_point() + ggtitle("ARI: Kmeans clustering (simple)") +
               ylim(0, 1),
             ggplot(data.frame(k = 1:20, ARI = kmeans_hard), aes(x = k, y = ARI)) +
               geom_line() + geom_point() + ggtitle("ARI: Kmeans clustering (hard)") +
               ylim(0, 1),
             ncol = 2)
```

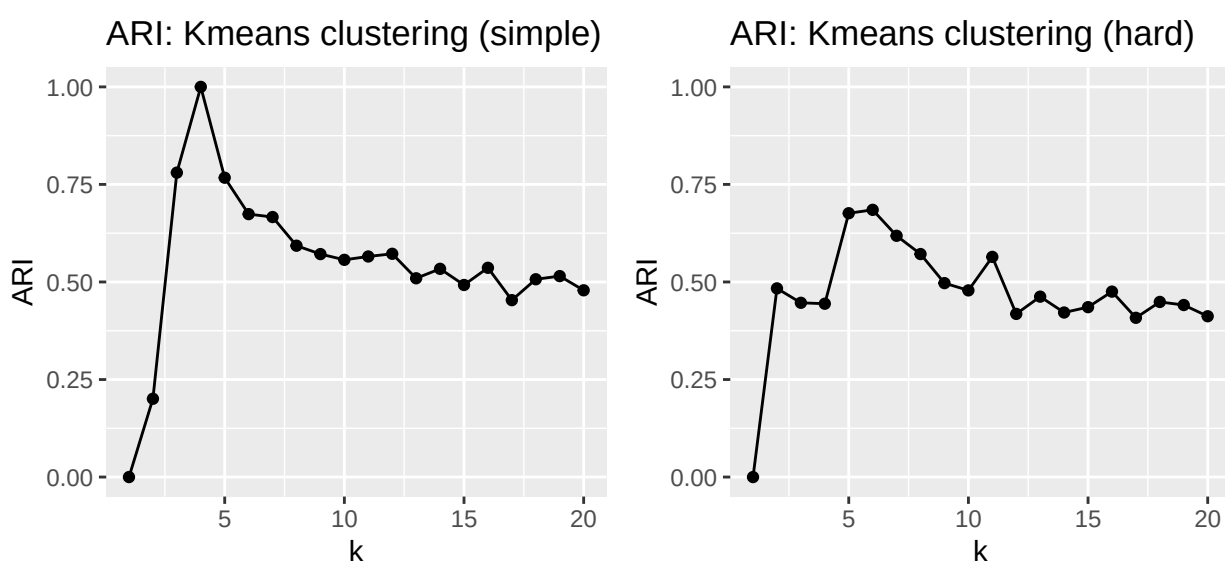

Figure 3: ARI scores for different number of clusters based on kmeans clustering.

```r
kout_simple <- kmeans(pca_simple$x, centers = 4)
sce_simple$Cluster_KM <- factor(kout_simple$cluster)
kout_hard <- kmeans(pca_hard$x, centers = 4)
sce_hard$Cluster_KM <- factor(kout_hard$cluster)
```

```r
grid.arrange(plotUMAP(sce_simple, colour_by = "Group", shape_by = "Cluster_KM") +
               ggtitle("UMAP visualization (simple)"),
             plotUMAP(sce_hard, colour_by = "Group", shape_by = "Cluster_KM") +
               ggtitle("UMAP visualization (hard)"),
             ncol = 2)
```

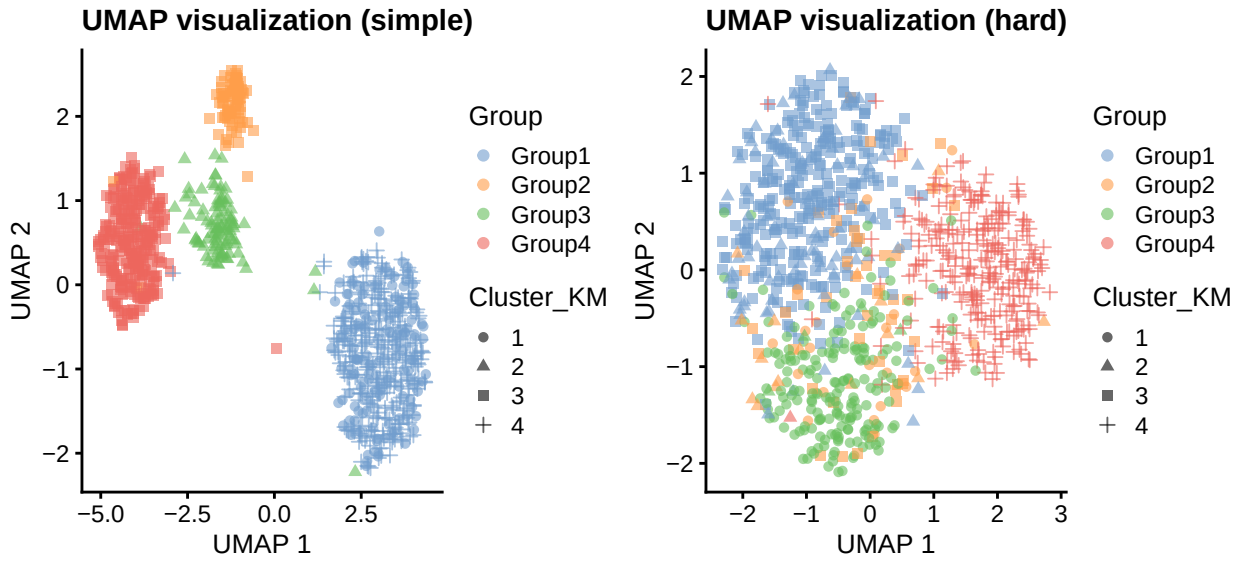

Figure 4: UMAP visualizations of both datasets based on kmeans clustering.

```r
(tab_simple <- table(sce_simple$Group, sce_simple$Cluster_KM))
```

```
##
##            1   2   3   4
##   Group1 134   0   0 339
##   Group2   0   0  86   0
##   Group3   0 102   0   0
##   Group4   0   0 285   0
```

```r
(tab_hard <- table(sce_hard$Group, sce_hard$Cluster_KM))
```

```
##
##            1   2   3   4
##   Group1   0  83 292   0
##   Group2  25  16  44   0
##   Group3 192   4   0   0
##   Group4   0   1   0 288
```

```r
solve_LSAP(tab_simple, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 4, 2 => 1, 3 => 2, 4 => 3
```

```r
solve_LSAP(tab_hard, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 3, 2 => 2, 3 => 1, 4 => 4
```

Hierarchical clustering on the other hand works based on distances, thus the euclidean distance matrix for the principal component analaysis matrix was calculated. As for the k-means analysis the optimal number of clusters were evaluated by computing the adjusted rand index. In comparison to k-means, hierarchical clustering could not detect the right amount of clusters for both datasets and overall achieved bad results for the second dataset (see Figure 5).

```r
# hierarchical clustering
hc_clustering <- function(sce, dist) {
  hc <- hclust(dist, method = "ward.D2")
  adj_rand <- foreach(k = 1:20, .combine = "c") %dopar% {
    tr <- cutree(hc, k = k)
    adjustedRandIndex(sce$Group, tr)
  }
  adj_rand
}

hc_simple <- hc_clustering(sce_simple, dist_simple)
hc_hard <- hc_clustering(sce_hard, dist_hard)
```

```r
grid.arrange(ggplot(data.frame(k = 1:20, ARI = hc_simple), aes(x = k, y = ARI)) +
               geom_line() + geom_point() + ggtitle("ARI: HClust clustering (simple)") +
               ylim(0, 1),
             ggplot(data.frame(k = 1:20, ARI = hc_hard), aes(x = k, y = ARI)) +
               geom_line() + geom_point() + ggtitle("ARI: HClust clustering (hard)") +
               ylim(0, 1),
             ncol = 2)
```

```r
hc_simple <- hclust(dist_simple, method = "ward.D2")
tr_simple <- cutree(hc_simple, k = 4)
hc_hard <- hclust(dist_hard, method = "ward.D2")
tr_hard <- cutree(hc_hard, k = 4)

sce_simple$Cluster_HC <- factor(tr_simple)
sce_hard$Cluster_HC <- factor(tr_hard)
```

```r
grid.arrange(plotUMAP(sce_simple, colour_by = "Group", shape_by = "Cluster_HC") +
               ggtitle("UMAP visualization (simple)"),
             plotUMAP(sce_hard, colour_by = "Group", shape_by = "Cluster_HC") +
               ggtitle("UMAP visualization (hard)"),
             ncol = 2)
```

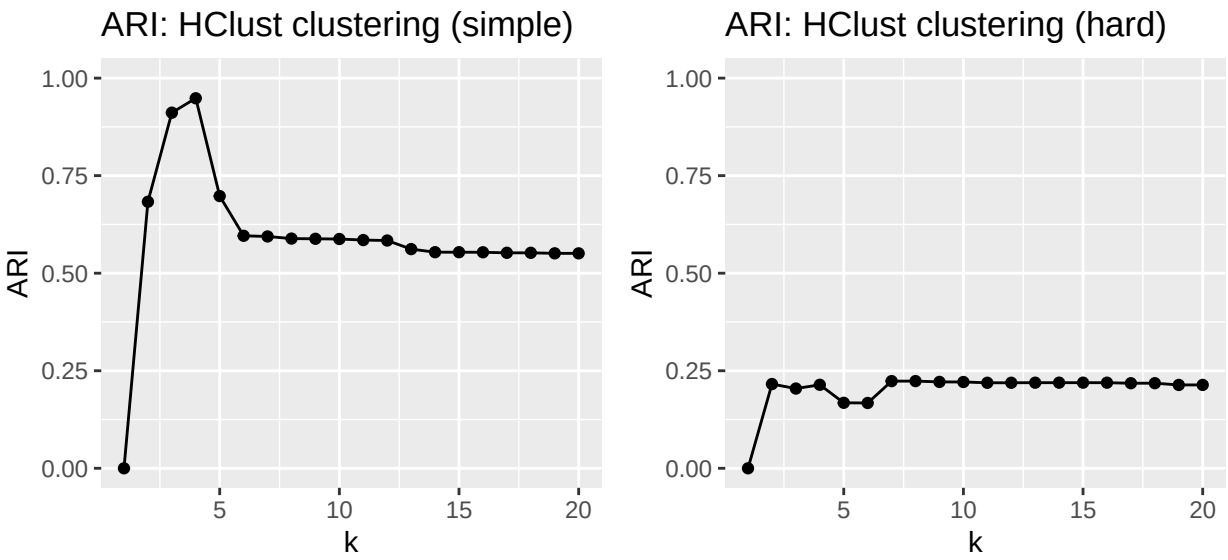

Figure 5: ARI scores for the two datasets using hierarchical clustering.

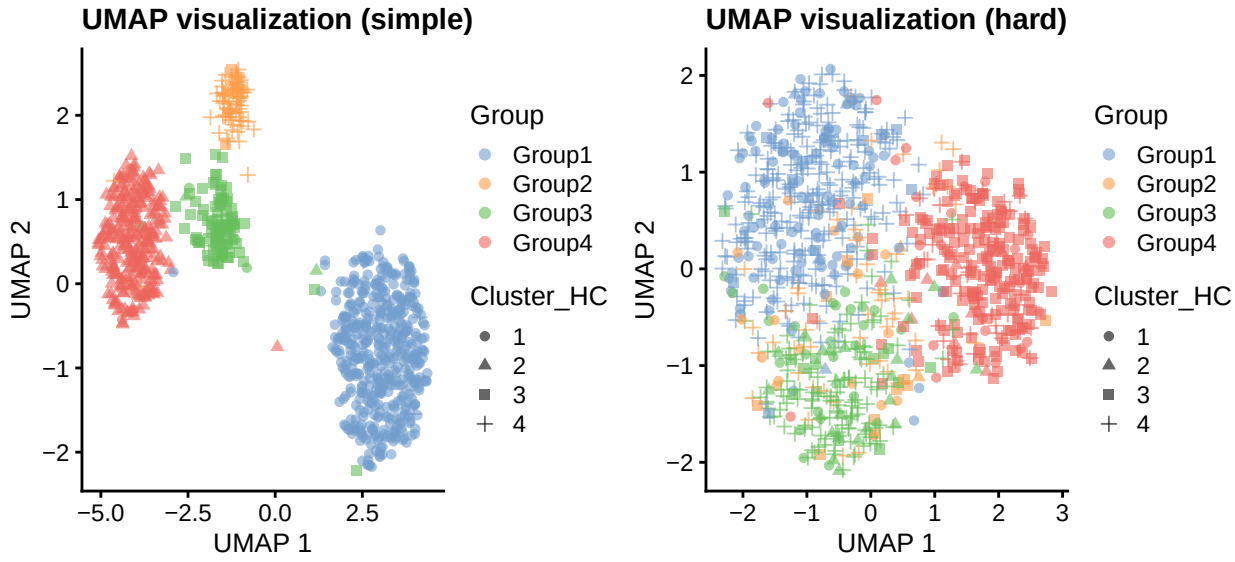

Figure 6: UMAP visualizations for both datasets based on hierarchic clustering.

```r
(tab_simple <- table(sce_simple$Group, sce_simple$Cluster_HC))
```

```
##
##            1   2   3   4
##   Group1 467   3   0   3
##   Group2   0   0   2  84
##   Group3   3   2  94   3
##   Group4   4 280   1   0
```

```r
(tab_hard <- table(sce_hard$Group, sce_hard$Cluster_HC))
```

```
##
##            1   2   3   4
##   Group1 106   6  12 251
##   Group2  17   2  11  55
##   Group3  26  23  13 134
##   Group4  25  11 205  48
```

```r
solve_LSAP(tab_simple, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 1, 2 => 4, 3 => 3, 4 => 2
```

```r
solve_LSAP(tab_hard, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 4, 2 => 1, 3 => 2, 4 => 3
```

3.4 Clustering - Seurat pipeline

To work with the Seurat package first a Seurat object has to be created, for that a wrapper function that interacts with the datasets was used. In contrast to the above used techniques Seurat calculates the dimensional reduction matrices by focusing on variable features, as opposed to integrating all available features.

```r
# load libraries
library(Seurat)

# create seurat objects from single cell experiment objects
seurat_simple <- as.Seurat(sce_simple)
seurat_hard <- as.Seurat(sce_hard)

# find variable features
seurat_simple <- FindVariableFeatures(seurat_simple, verbose = FALSE)
seurat_hard <- FindVariableFeatures(seurat_hard, verbose = FALSE)

# scale features before using dimensional reduction techinques
dimension_reduction_seurat <- function(sce_seurat) {
  sce_seurat <- ScaleData(sce_seurat, features = rownames(sce_seurat),
                          verbose = FALSE)
```

```r
sce_seurat <- RunPCA(sce_seurat, features = VariableFeatures(sce_seurat),
                     verbose = FALSE)
sce_seurat <- RunUMAP(sce_seurat, features = VariableFeatures(sce_seurat),
                      verbose = FALSE)
sce_seurat
}

seurat_simple <- dimension_reduction_seurat(seurat_simple)
seurat_hard <- dimension_reduction_seurat(seurat_hard)
```

To cluster the datasets, Seurat calculates the shared nearest neighbors using the Jaccard index to measure the neighborhood overlap between every cell. The ideal choice of resolution was implemented by calculating the maximum adjusted rand index for resolution values between zero and one. The result of this calculation can be seen in Figure 7.

```r
# load libraries
library(dplyr)

# seurat clustering
seurat_clustering <- function(sce_seurat, resolution) {
  # find nearest neighbour
  sce_seurat <- FindNeighbors(sce_seurat, verbose = FALSE)

  # find clusters
  sce_seurat <- FindClusters(sce_seurat, resolution = resolution, verbose = FALSE)

  sce_seurat
}

resolution_simple <- foreach(r = seq(0, 1, by = 0.1), .combine = "c") %dopar% {
  seurat_simple <- seurat_clustering(seurat_simple, resolution = r)
  adjustedRandIndex(seurat_simple$Group, seurat_simple$seurat_clusters)
}

resolution_hard <- foreach(r = seq(0, 1, by = 0.1), .combine = "c") %dopar% {
  seurat_hard <- seurat_clustering(seurat_hard, resolution = r)
  adjustedRandIndex(seurat_hard$Group, seurat_hard$seurat_clusters)
}
```

```r
grid.arrange(ggplot(data.frame(r = seq(0, 1, by = 0.1), ARI = resolution_simple),
                    aes(x = r, y = ARI)) + geom_line() + geom_point() +
               ggtitle("ARI: Seurat clustering (simple)") + ylim(0, 1),
             ggplot(data.frame(r = seq(0, 1, by = 0.1), ARI = resolution_hard),
                    aes(x = r, y = ARI)) + geom_line() + geom_point() +
               ggtitle("ARI: Seurat clustering (hard)") + ylim(0, 1),
             ncol = 2)
```

To validate whether the clustering was good the marker genes can be identified, which are differentially expressed for the given clusters as can be seen in Figure 8 and 9. These marker genes were then used to plot gene activity plots which approximate the real clustering quite well for both datasets (see Figure 10 and Figure 11).

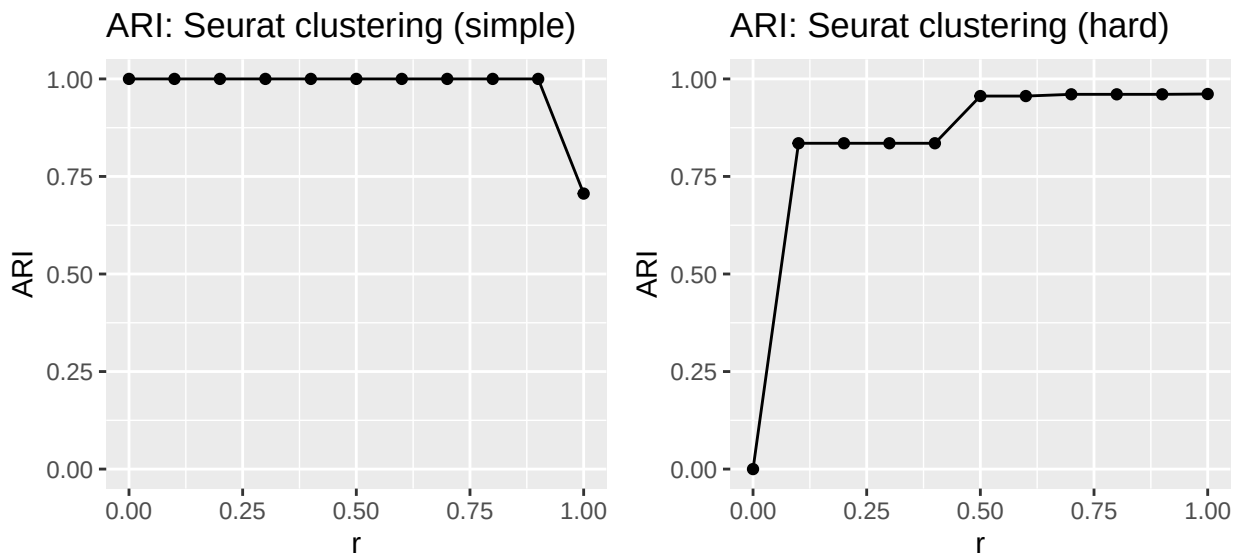

Figure 7: ARI scores for the two datasets using Seurat.

```
seurat_simple <- seurat_clustering(seurat_simple, resolution = 0.5)
seurat_hard <- seurat_clustering(seurat_hard, resolution = 0.8)

seurat_markers <- function(sce_seurat) {
  # find marker genes in clusters using ROC to evaluate best markers
  cluster_markers <- FindAllMarkers(sce_seurat, only.pos = TRUE,
                                    min.pct = 0.25, logfc.threshold = 0.25,
                                    test.use = "roc", verbose = FALSE)
  top_markers <- cluster_markers %>% group_by(cluster) %>%
    top_n(n = 2, wt = power) %>% top_n(n = 1, wt = avg_diff)
  top_markers
}

markers_simple <- seurat_markers(seurat_simple)
markers_hard <- seurat_markers(seurat_hard)
```

```
# violin plot to show expression in the marker genes by clusters
VlnPlot(seurat_simple, features = markers_simple$gene, group.by = "Group", pt.size = 0) +
  ggtitle("Marker genes for each seurat cluster (simple)")
```

```
# violin plot to show expression in the marker genes by clusters
VlnPlot(seurat_hard, features = markers_hard$gene, group.by = "Group", pt.size = 0) +
  ggtitle("Marker genes for each seurat cluster (hard)")
```

```
# feature plot to show activity in groups to compare with above plot
FeaturePlot(seurat_simple, features = markers_simple$gene)
```

```
# feature plot to show activity in groups to compare with above plot
FeaturePlot(seurat_hard, features = markers_hard$gene)
```

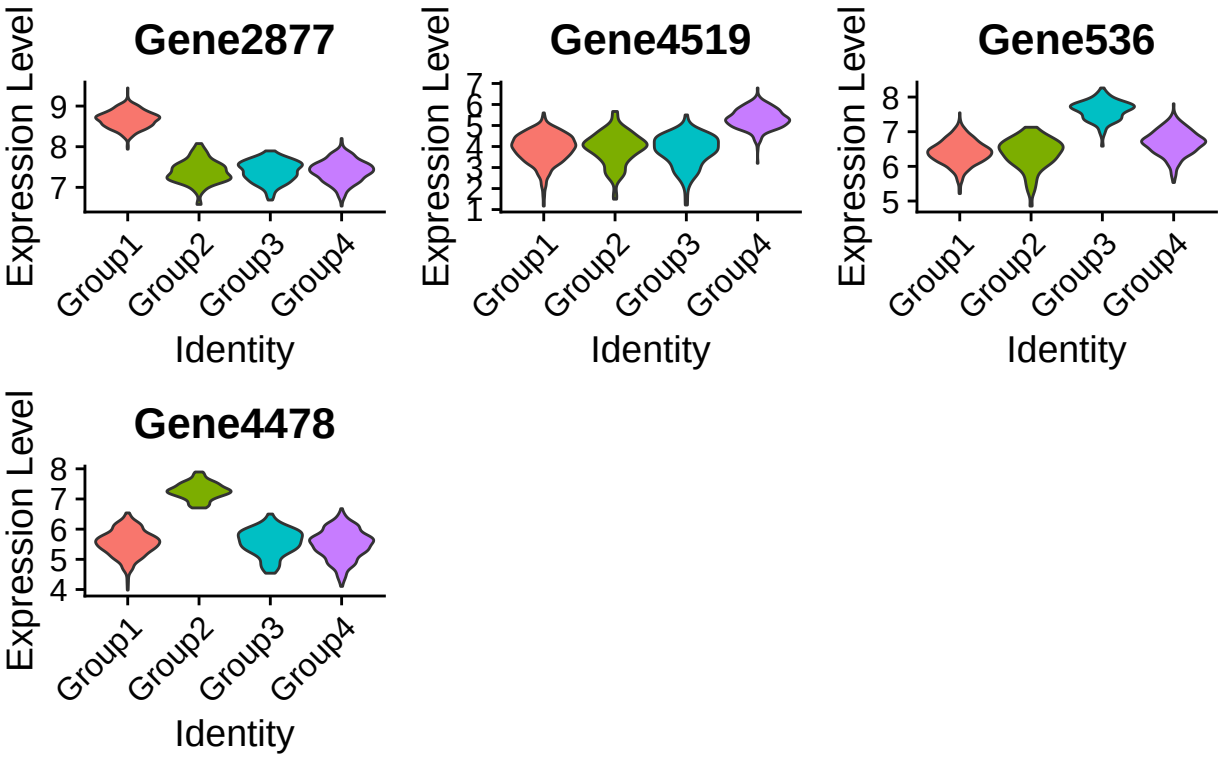

Figure 8: Violin plots to show expression in the marker genes of the simple dataset by Seurat clusters.

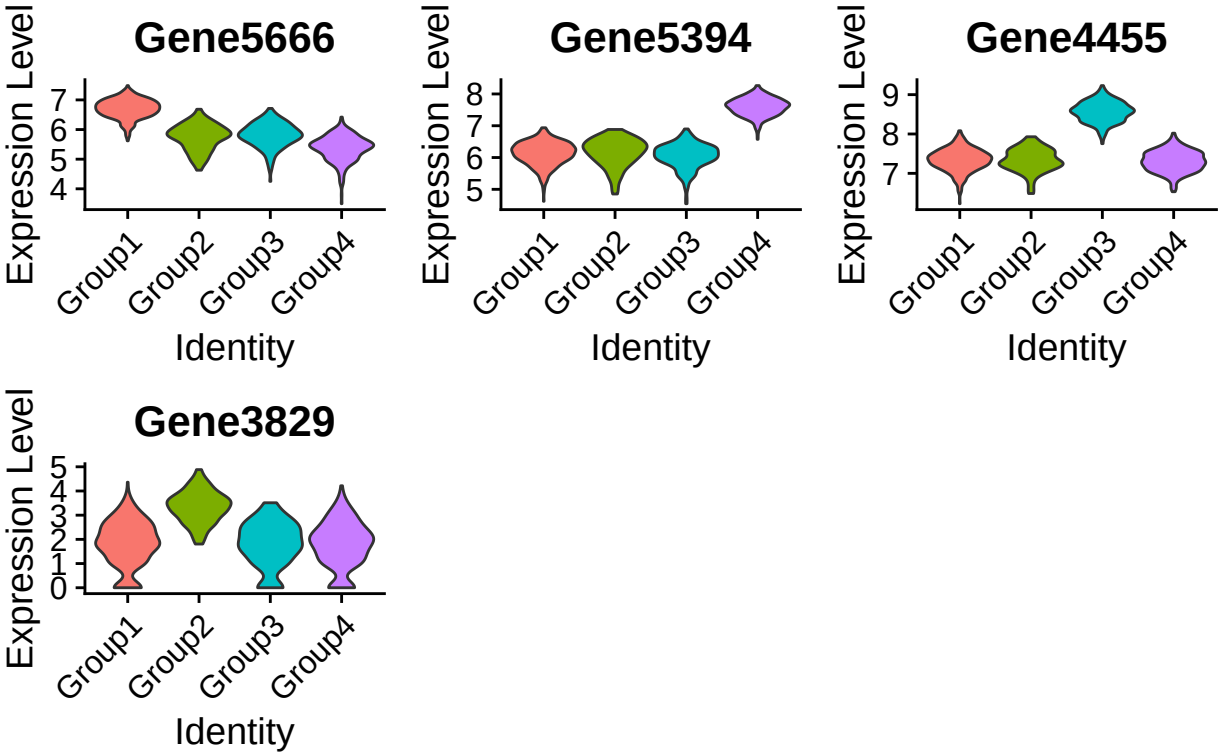

Figure 9: Violin plots to show expression in the marker genes of the hard dataset by Seurat clusters.

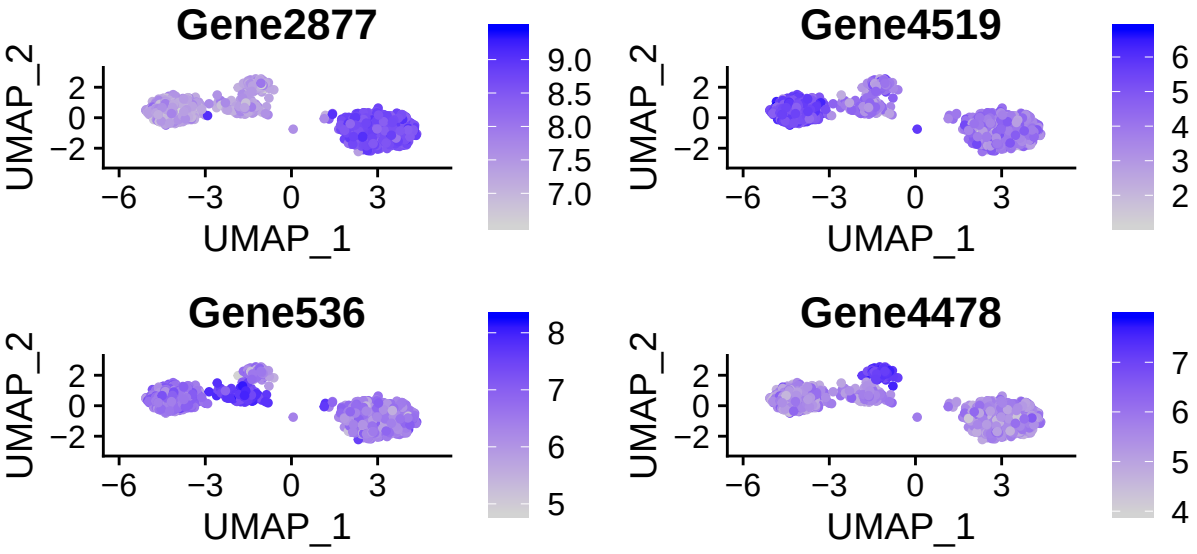

Figure 10: Feature plot of marker gene activity in Seurat clusters of the simple dataset.

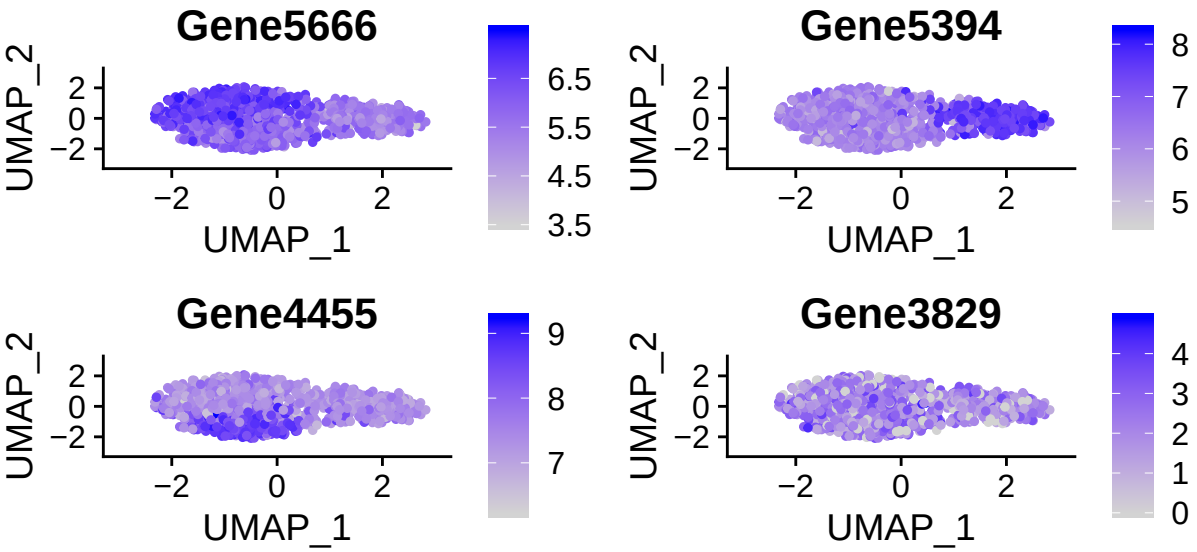

Figure 11: Feature plot of marker gene activity in Seurat clusters of the hard dataset.

```r
# Rename clusters to real names
(tab_simple <- table(seurat_simple$Group, seurat_simple$seurat_clusters))
```

```
##
##            0   1   2   3
##   Group1 473   0   0   0
##   Group2   0   0   0  86
##   Group3   0   0 102   0
##   Group4   0 285   0   0
```

```r
(tab_hard <- table(seurat_hard$Group, seurat_hard$seurat_clusters))
```

```
##
##            0   1   2   3
##   Group1 373   0   0   2
##   Group2  13   0   1  71
##   Group3   0   0 194   2
##   Group4   0 289   0   0
```

```r
solve_LSAP(tab_simple, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 1, 2 => 4, 3 => 3, 4 => 2
```

```r
solve_LSAP(tab_hard, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 1, 2 => 4, 3 => 3, 4 => 2
```

```r
seurat_simple <- RenameIdents(seurat_simple, "0" = "Group1",
                              "1" = "Group4", "2" = "Group3",
                              "3" = "Group2")
seurat_hard <- RenameIdents(seurat_hard, "0" = "Group1",
                             "1" = "Group4", "2" = "Group3",
                             "3" = "Group2")
```

The resulting clusters from the Seurat pipeline can be visualized in two dimensions using a UMAP-dimension reduction as has been done in Figure 12. On the left side of the plot, one can see the clusters from the simple dataset and on the right side the clustering of the hard dataset. It can clearly be seen that the clustering works very well for the simple dataset whereas the representation of the clusters from the harder dataset shows some overlaps. Cell groups 1, 3 and 4 are more or less easy to spot but the cluster from group 2 has large intersections with the clusters from group 3 and group 1.

```r
grid.arrange(DimPlot(seurat_simple, reduction = "umap",
                     shape.by = "seurat_clusters", label = TRUE) + NoLegend(),
             DimPlot(seurat_hard, reduction = "umap",
                     shape.by = "seurat_clusters", label = TRUE) + NoLegend(),
             ncol = 2)
```

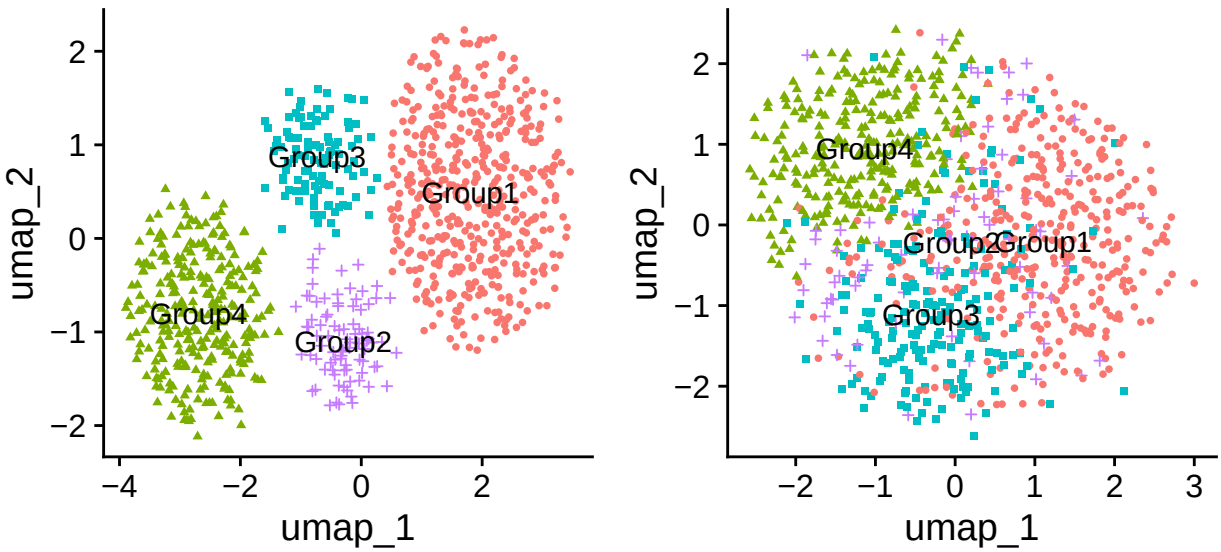

Figure 12: Left: UMAP dimensional reduction plots for the simple dataset. ; Right: UMAP dimensional reduction plots for the hard dataset.

3.5 Clustering - SC3 pipeline

As an input for the SC3 pipeline an object of class SingleCellExperiment is needed. Since these are already available from the analysis above, they can be reused and the respective slots can be filled by the SC3 pipeline. What is furthermore interesting is that the SC3-package provides a function to estimate the optimal number of clusters which according to the formula in the package is 3 in both datasets. However, the packages are simulated and the true number of clusters is 4 which is why at this point the decision was made to compute the clustering only for 3 and 4 clusters since the various k-means clusterings performed in the SC3-pipeline are quite time consuming.

```
# load libraries
library(SC3)

#set feature_symbols
rowData(sce_simple)$feature_symbol <- rownames(sce_simple)
rowData(sce_hard)$feature_symbol <- rownames(sce_hard)

# defining parameters needed for clustering
sce_simple <- sc3_prepare(sce_simple, rand_seed = seed)
sce_hard <- sc3_prepare(sce_hard, rand_seed = seed)

# estimate the optimal number of clusters in the dataset
sce_simple <- sc3_estimate_k(sce_simple)
sce_hard <- sc3_estimate_k(sce_hard)

c(paste("Proposed number of clusters for simple dataset:",
      metadata(sce_simple)$sc3$k_estimation)
  , paste("Proposed number of clusters for hard dataset:",
      metadata(sce_simple)$sc3$k_estimation))
```

```
## [1] "Proposed number of clusters for simple dataset: 3"
## [2] "Proposed number of clusters for hard dataset: 3"
```

```r
# calculate distance matrices
sce_simple <- sc3_calc_dists(sce_simple)
sce_hard <- sc3_calc_dists(sce_hard)

# applying PCA and graph Laplacian to distance matrices
sce_simple <- sc3_calc_transfs(sce_simple)
sce_hard <- sc3_calc_transfs(sce_hard)

# performing kmeans on transformed distance matrices
# set nr. of clusters to 3 and 4 since 4 is true number and 3 was proposed
set.seed(seed)
sce_simple <- sc3_kmeans(sce_simple, ks = 3:4)
```

```r
sce_hard <- sc3_kmeans(sce_hard, ks = 3:4)
```

```r
# calculate consensus matrices for each k
sce_simple <- sc3_calc_consens(sce_simple)
sce_hard <- sc3_calc_consens(sce_hard)

# calculate DE genes, marker genes and cell outliers based on
# consensus clustering
sce_simple <- sc3_calc_biology(sce_simple, ks = 3:4)
sce_hard <- sc3_calc_biology(sce_hard, ks = 3:4)
```

As a first step of the analysis, it might be interesting to look at the ARI-scores for the clustering since the proposed number of clusters for both datasets is 3 instead of 4.

```r
# Comparing ARI-scores of clusterings with k = c(3,4) and true groups
paste("ARI-score for k = 3 on simple dataset:",
  round(adjustedRandIndex(sce_simple$Group, sce_simple$sc3_3_clusters), 3))
```

```
## [1] "ARI-score for k = 3 on simple dataset: 0.958"
```

```r
paste("ARI-score for k = 3 on hard dataset:",
  round(adjustedRandIndex(sce_hard$Group, sce_hard$sc3_3_clusters), 3))
```

```
## [1] "ARI-score for k = 3 on hard dataset: 0.782"
```

```r
paste("ARI-score for k = 4 on simple dataset:",
  round(adjustedRandIndex(sce_simple$Group, sce_simple$sc3_4_clusters), 3))
```

```
## [1] "ARI-score for k = 4 on simple dataset: 1"
```

```r
paste("ARI-score for k = 4 on hard dataset:",
  round(adjustedRandIndex(sce_hard$Group, sce_hard$sc3_4_clusters), 3))
```

```
## [1] "ARI-score for k = 4 on hard dataset: 0.775"
```

Looking at the two datasets one can see that the simple dataset has a higher ARI-score with 4 clusters than with 3 clusters. However, the harder to distinguish dataset has a slightly lower ARI-score with 4 clusters than with 3. This is a little surprising but overall the scores indicate that the clustering with the SC3-pipeline works better than the simple k-means approach or the hierarchical clustering. In a next step, the resulting clusters are relabeled in order to match the true group association of the cells as far as possible and marker genes are identified by using the area under the ROC-curve which has conveniently been calculated for both datasets and both numbers of clusters.

```
# relabel clusters to match true group number
relab_sce_simple <- sce_simple
relab_sce_hard <- sce_hard

table(relab_sce_simple$Group, relab_sce_simple$sc3_3_clusters)
```

```
##
##            1   2   3
##   Group1 473   0   0
##   Group2   0   0  86
##   Group3   0   0 102
##   Group4   0 285   0
```

```
idx2 <- which(sce_simple$sc3_3_clusters == 2)
idx3 <- which(sce_simple$sc3_3_clusters == 3)
relab_sce_simple$sc3_3_clusters[idx2] <- 3
relab_sce_simple$sc3_3_clusters[idx3] <- 2

table(relab_sce_hard$Group, relab_sce_hard$sc3_3_clusters)
```

```
##
##            1   2   3
##   Group1   1 349  25
##   Group2   1  43  41
##   Group3   0   2 194
##   Group4 287   0   2
```

```
idx1 <- which(relab_sce_hard$sc3_3_clusters == 1)
idx3 <- which(relab_sce_hard$sc3_3_clusters == 3)
relab_sce_hard$sc3_3_clusters[idx1] <- 3
relab_sce_hard$sc3_3_clusters[idx3] <- 1

table(relab_sce_simple$Group, relab_sce_simple$sc3_4_clusters)
```

```
##
##            1   2   3   4
##   Group1 473   0   0   0
##   Group2   0   0   0  86
##   Group3   0   0 102   0
##   Group4   0 285   0   0
```

```
idx2 <- which(sce_simple$sc3_4_clusters == 2)
idx4 <- which(sce_simple$sc3_4_clusters == 4)
relab_sce_simple$sc3_4_clusters[idx2] <- 4
```

```r
relab_sce_simple$sc3_4_clusters[idx4] <- 2

(tab_hard <- table(relab_sce_hard$Group, relab_sce_hard$sc3_4_clusters))
```

```
##
##            1   2   3   4
##   Group1   1   0 358  16
##   Group2   1   0  51  33
##   Group3   1   0   6 189
##   Group4   7 278   1   3
```

```r
solve_LSAP(tab_hard, maximum = TRUE)
```

```
## Optimal assignment:
## 1 => 3, 2 => 1, 3 => 4, 4 => 2
```

```r
idx1 <- which(relab_sce_hard$sc3_4_clusters == 1)
idx2 <- which(relab_sce_hard$sc3_4_clusters == 2)
idx3 <- which(relab_sce_hard$sc3_4_clusters == 3)
idx4 <- which(relab_sce_hard$sc3_4_clusters == 4)
relab_sce_hard$sc3_4_clusters[idx1] <- 4
relab_sce_hard$sc3_4_clusters[idx2] <- 2
relab_sce_hard$sc3_4_clusters[idx3] <- 1
relab_sce_hard$sc3_4_clusters[idx4] <- 3

# finding marker genes
sc3_marker <- function(sce, n_cluster){

  # get dataframe from rowData of sce
  rd <- rowData(sce)
  rd <- na.omit(rd)
  clust_string <- paste("sc3_", n_cluster, sep = "")
  rd <- cbind("marker_gene" = rownames(rd),
              as.data.frame(rd[ , grep(clust_string, colnames(rd))]))
  rd <- rd[order(rd[, 4], decreasing = T),]
  rownames(rd) <- c()

  # find marker genes
  idx <- numeric(n_cluster)
  for(i in 1:n_cluster){
    idx[i] <- which(rd[,2] == i, arr.ind = T)[1]
  }
  return(rd[idx, 1:4])
}

# marker genes for k = 3 on the simple dataset
(simple_mark_3_clust <- sc3_marker(relab_sce_simple, 3))
```

```
##    marker_gene sc3_3_markers_clusts sc3_3_markers_padj sc3_3_markers_auroc
## 1     Gene1445                    1       9.964032e-93           0.8908680
## 3     Gene7288                    2       3.127231e-50           0.8165486
## 13    Gene6188                    3       2.715988e-22           0.7424037
```

```r
# marker genes for k = 4 on the simple dataset
(simple_mark_4_clust <- sc3_marker(relab_sce_simple, 4))
```

```
##    marker_gene sc3_4_markers_clusts sc3_4_markers_padj sc3_4_markers_auroc
## 3     Gene1445                    1       9.964032e-93           0.8908680
## 10    Gene7288                    2       3.127231e-50           0.8165486
## 6     Gene5372                    3       4.827021e-27           0.8341058
## 1     Gene3903                    4       9.781331e-41           0.9569091
```

```r
# marker genes for k = 3 on the hard dataset
(hard_mark_3_clust <- sc3_marker(relab_sce_hard, 3))
```

```
##   marker_gene sc3_3_markers_clusts sc3_3_markers_padj sc3_3_markers_auroc
## 1    Gene5898                    1       1.285757e-62           0.8506915
## 6    Gene4559                    2       2.571615e-35           0.7488922
## 3    Gene9464                    3       1.525418e-40           0.7921161
```

```r
# marker genes for k = 4 on the hard dataset
(hard_mark_4_clust <- sc3_marker(relab_sce_hard, 4))
```

```
##    marker_gene sc3_4_markers_clusts sc3_4_markers_padj sc3_4_markers_auroc
## 1     Gene6386                    1       1.901982e-01           0.8537968
## 2     Gene5898                    2       6.499350e-61           0.8498916
## 20    Gene4559                    3       1.233900e-33           0.7415843
## 4     Gene9464                    4       7.722978e-42           0.8045313
```

Looking at the identified marker genes one can see that the simple dataset shows for both - 3 and 4 clusters - rather good marker genes whereas it is striking that the clustering with 4 clusters in the hard dataset identifies a marker gene for cluster 2 with an adjusted p-value of 0.94. In order to check why this is the case it might be helpful to look at the gene expression of these genes.

```r
# plot marker gene expression for different clusters in each dataset with k = 3
plotExpression(relab_sce_simple, features = as.character(simple_mark_3_clust$marker_gene),
               x = "sc3_3_clusters", colour_by = "sc3_3_clusters",
               xlab = "Cluster Nr.") +
  ggtitle("Marker gene expression for sc3-clusters with k = 3 (simple dataset)")
```

```r
plotExpression(relab_sce_hard, features = as.character(hard_mark_3_clust$marker_gene),
               x = "sc3_3_clusters", colour_by = "sc3_3_clusters",
               xlab = "Cluster Nr.") +
  ggtitle("Marker gene expression for sc3-clusters with k = 3 (hard dataset)")
```

```r
# plot marker gene expression for different clusters in each dataset with k = 4
plotExpression(relab_sce_simple, features = as.character(simple_mark_4_clust$marker_gene),
               x = "sc3_4_clusters", colour_by = "sc3_4_clusters",
               xlab = "Cluster Nr.") +
  ggtitle("Marker gene expression for sc3-clusters with k = 4 (simple dataset)")
```

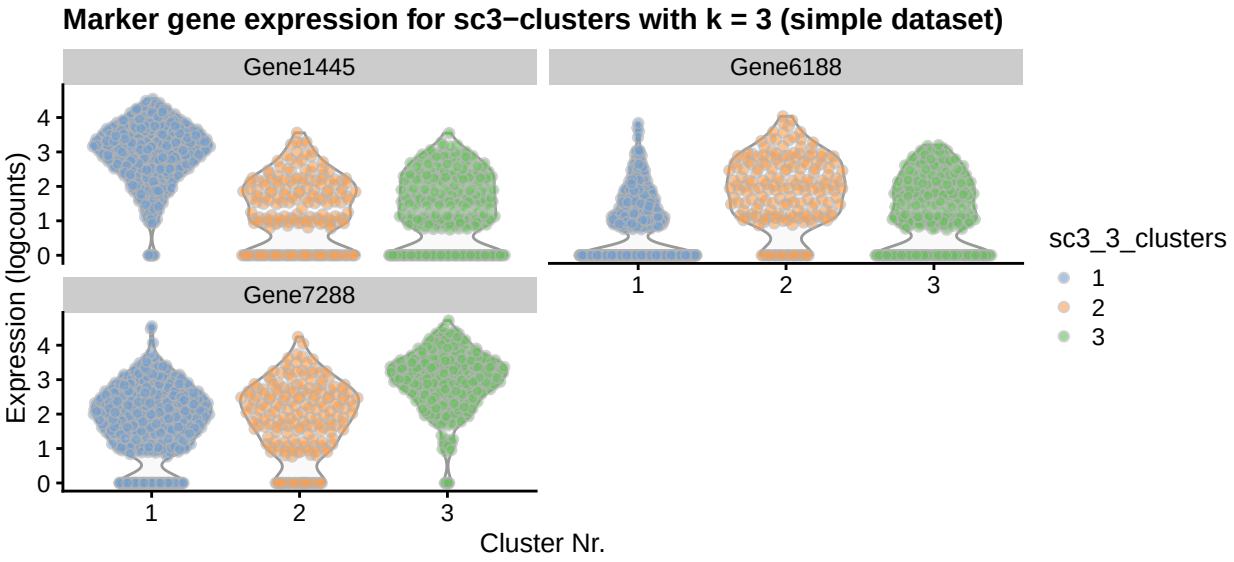

Figure 13: Marker gene expression in SC3 clusters with k = 3 on the simple dataset.

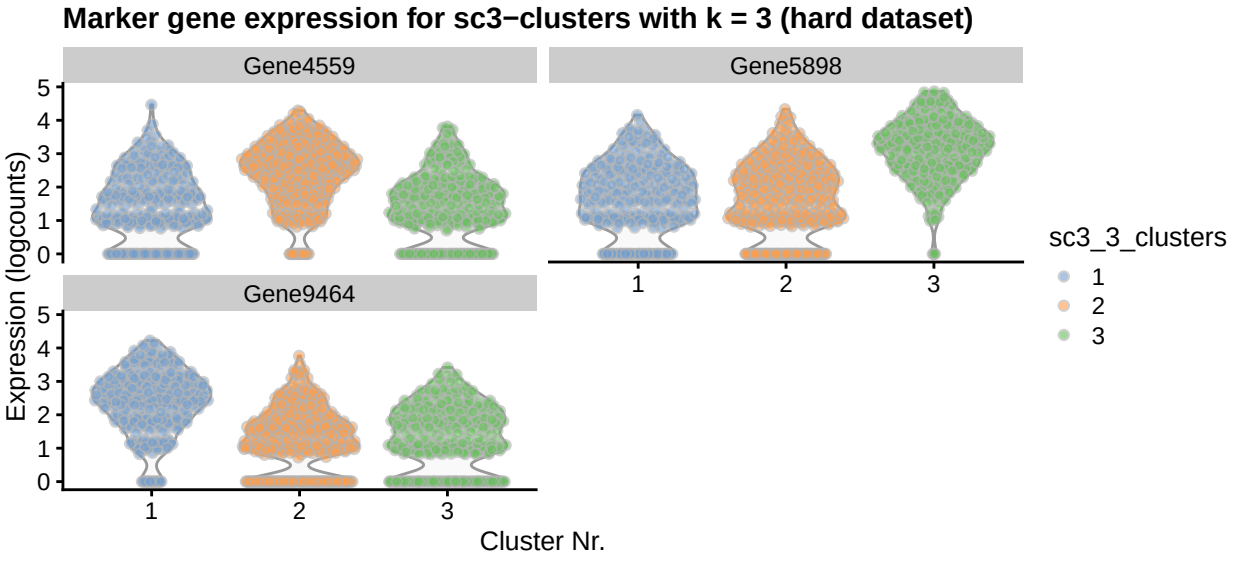

Figure 14: Marker gene expression in SC3 clusters with k = 3 on the hard dataset.

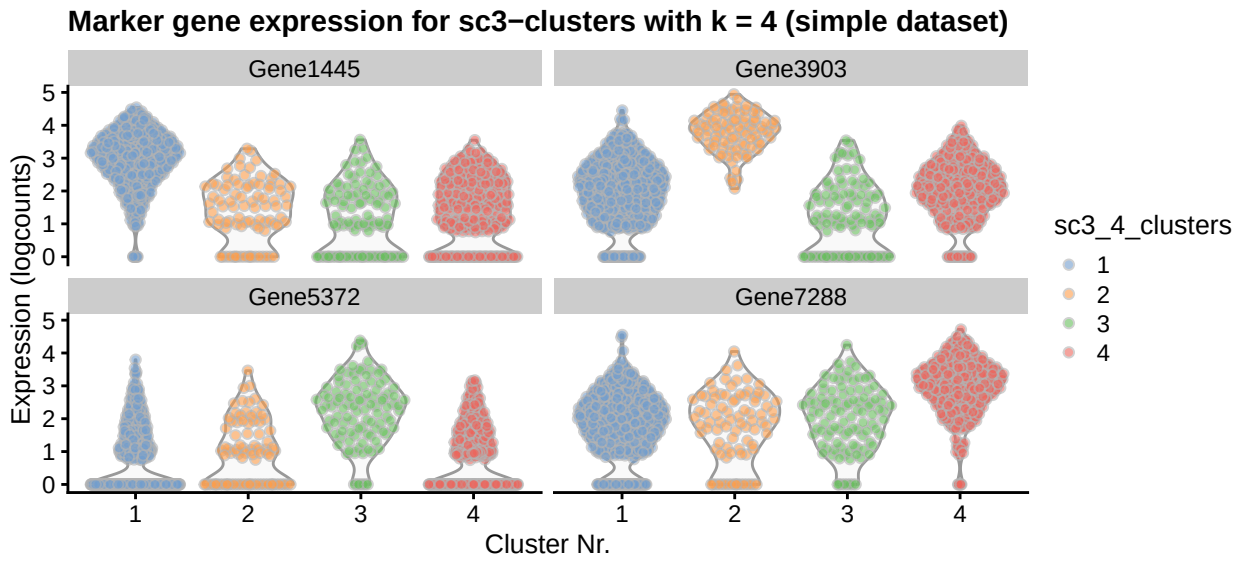

Figure 15: Marker gene expression in SC3 clusters with k = 4 on the simple dataset.

```
plotExpression(relab_sce_hard, features = as.character(hard_mark_4_clust$marker_gene),
               x = "sc3_4_clusters", colour_by = "sc3_4_clusters",
               xlab = "Cluster Nr.") +
  ggtitle("Marker gene expression for sc3-clusters with k = 4 (hard dataset)")
```

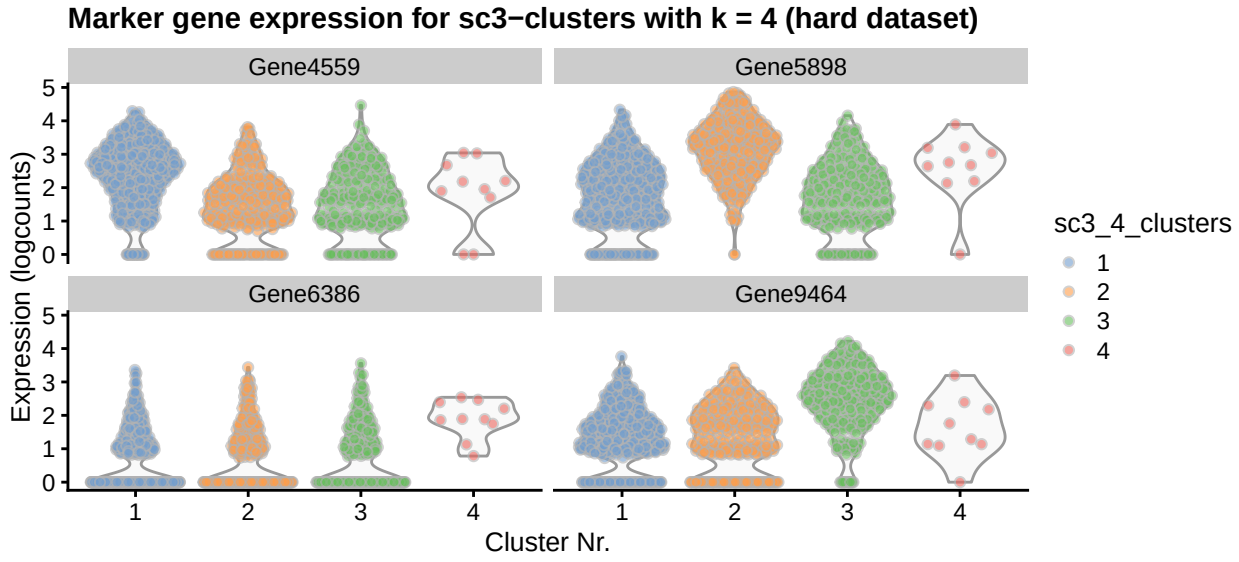

Figure 16: Marker gene expression in SC3 clusters with k = 4 on the hard dataset.

In Figures 13, 14, 15 and 16 the gene expression of the identified marker genes is visualized. According to the plots, the genes identified as markers seem to fulfill their purpose quite well in general. However, the violin-plots in Figure 16 show that the differential expression is harder to spot than in the other three cases. One can also see why the adjusted p-value of the marker gene for cluster 2 is so high. The reason for this is just that there are very few observations in that cluster.

Similar to the other methods used in this project, it might be interesting to see if these clusters can be

visualized through dimensional reduction techniques. Figure 17 and Figure 18 show that the clustering worked very well for the simple dataset. In the plots refering to the hard dataset however, it can be seen that the clustering with three clusters worked quite well but with four clusters SC3 struggles quite a bit to find cluster number 2. This is not surprising after seeing the violin plots above and the very high adjusted p-value for the marker gene for cluster 2 on the hard dataset.

```
# UMAP dimensional reduction for 3 clusters
library(grid)
grid.arrange(plotUMAP(relab_sce_simple,
                      colour_by = "sc3_3_clusters",
                      shape_by = "Group",),
             plotUMAP(relab_sce_hard, colour_by = "sc3_3_clusters",
                      shape_by = "Group"),
             ncol = 2, top = textGrob("UMAP-plot with k = 3", gp=gpar(fontsize=20)))
```

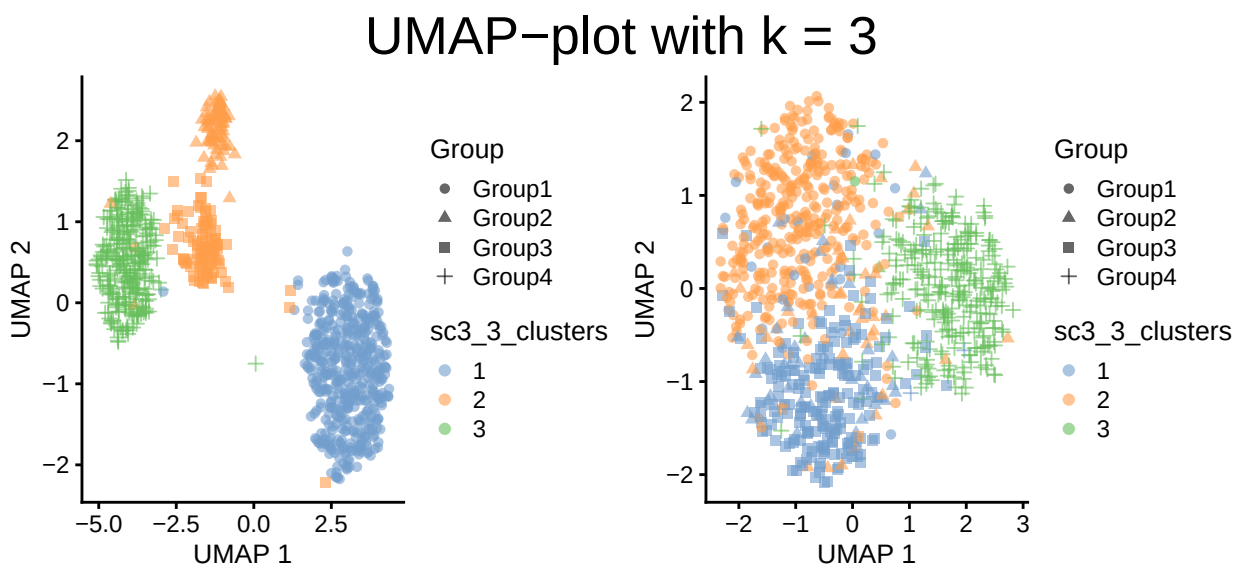

Figure 17: Left: UMAP-plot for the simple dataset; Right: UMAP-plot for the hard dataset.

```
# UMAP dimensional reduction for 4 clusters
grid.arrange(plotUMAP(relab_sce_simple,
                      colour_by = "sc3_4_clusters",
                      shape_by = "Group",),
             plotUMAP(relab_sce_hard, colour_by = "sc3_4_clusters",
                      shape_by = "Group"),
             ncol = 2, top = textGrob("UMAP-plot with k = 4", gp=gpar(fontsize=20)))
```

4 Conclusion

K-means clustering on the log-counts works very well for the simple dataset as it achieves a perfect ARI-score, on the second dataset the score drops but still remains in a good range. Hierarchical clustering on the other hand only seems to work well on the simple dataset, whereas on the hard dataset good clustering was not achieved. Contrary to the k-means clustering and the hierarchical clustering, the Seurat pipeline works excellent as it achieves almost perfect scores for both datasets. Also, in comparison to the SC3

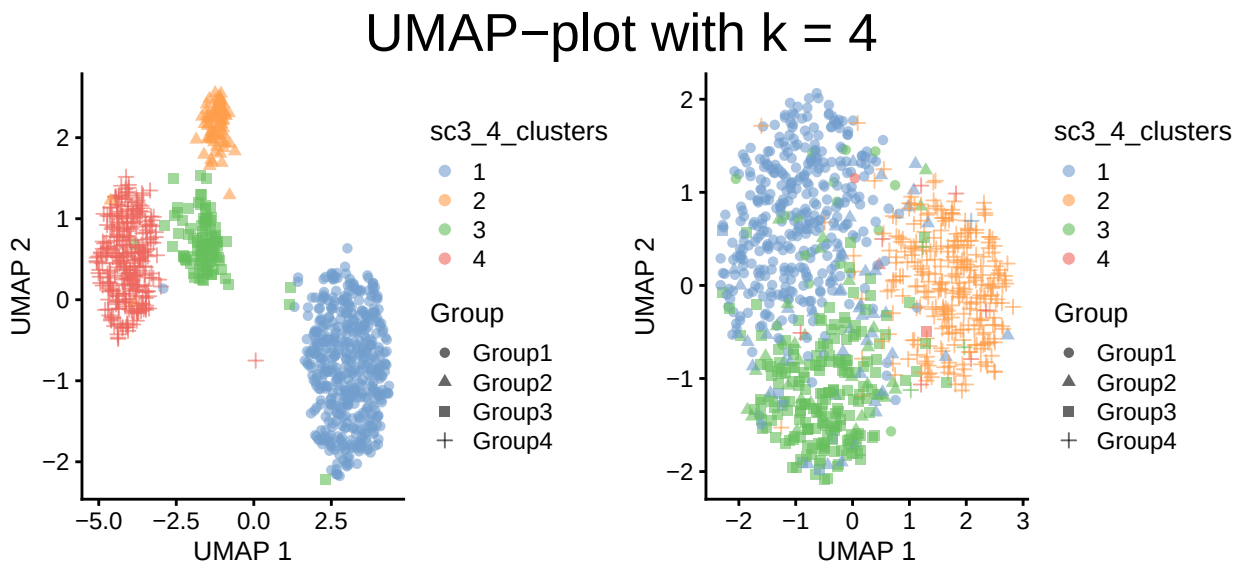

Figure 18: Left: UMAP-plot for the simple dataset; Right: UMAP-plot for the hard dataset.

clustering Seurat yields the higher ARI-values - especially for the hard dataset - and therefore works better for clustering purposes on these two datasets. Nevertheless, SC3 outperformed both the simple k-means approach and the hierarchical clustering even though its estimation of the number of clusters was not correct for our two datasets. In conclusion, it can be said that both the graph-based clustering and the consensus-clustering worked superior in comparison to both k-means and hiearchial clustering, especially when clusters are not obviously detectable. However, it is important during such analyses to be aware that some of the options might not work the way one might expect as was the case here with the mentioned problem that SC3 proposed too few clusters.

Session information

```r
# Session information for reproducibility
BiocManager::version()
```

```
## [1] '3.10'
```

```r
sessionInfo()
```

```
## R version 3.6.2 (2019-12-12)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Linux Mint 19.3
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.7.1
## LAPACK: /home/joel/.local/share/r-miniconda/envs/r-reticulate/lib/libmkl_rt.so
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
```

```
##  [5] LC_MONETARY=de_CH.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=de_CH.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=de_CH.UTF-8 LC_IDENTIFICATION=C
##
## attached base packages:
##  [1] grid      stats4    parallel  stats     graphics  grDevices utils
##  [8] datasets  methods   base
##
## other attached packages:
##  [1] SC3_1.14.0                  dplyr_0.8.3
##  [3] Seurat_3.1.2                mclust_5.4.5
##  [5] clue_0.3-57                 doParallel_1.0.15
##  [7] iterators_1.0.12            foreach_1.4.7
##  [9] gridExtra_2.3               scater_1.14.6
## [11] ggplot2_3.2.1               splatter_1.10.0
## [13] SingleCellExperiment_1.8.0  SummarizedExperiment_1.16.1
## [15] DelayedArray_0.12.1         BiocParallel_1.20.1
## [17] matrixStats_0.55.0          Biobase_2.46.0
## [19] GenomicRanges_1.38.0        GenomeInfoDb_1.22.0
## [21] IRanges_2.20.1              S4Vectors_0.24.1
## [23] BiocGenerics_0.32.0
##
## loaded via a namespace (and not attached):
##   [1] reticulate_1.14          R.utils_2.9.2            tidyselect_0.2.5
##   [4] htmlwidgets_1.5.1        Rtsne_0.15               munsell_0.5.0
##   [7] codetools_0.2-16         mutoss_0.1-12            ica_1.0-2
##  [10] future_1.15.1            withr_2.1.2              colorspace_1.4-1
##  [13] highr_0.8                knitr_1.26               rstudioapi_0.10
##  [16] ROCR_1.0-7               robustbase_0.93-5        gbRd_0.4-11
##  [19] listenv_0.8.0            Rdpack_0.11-1            labeling_0.3
##  [22] GenomeInfoDbData_1.2.2   mnormt_1.5-5             farver_2.0.1
##  [25] pheatmap_1.0.12          vctrs_0.2.1              TH.data_1.0-10
##  [28] xfun_0.11                R6_2.4.1                 ggbeeswarm_0.6.0
##  [31] rsvd_1.0.2               locfit_1.5-9.1           bitops_1.0-6
##  [34] assertthat_0.2.1         promises_1.1.0           SDMTools_1.1-221.2
##  [37] scales_1.1.0             multcomp_1.4-11          beeswarm_0.2.3
##  [40] gtable_0.3.0             npsurv_0.4-0             globals_0.12.5
##  [43] sandwich_2.5-1           rlang_0.4.2              zeallot_0.1.0
##  [46] splines_3.6.2            lazyeval_0.2.2           checkmate_1.9.4
##  [49] BiocManager_1.30.10      yaml_2.2.0               reshape2_1.4.3
##  [52] backports_1.1.5          httpuv_1.5.2             tools_3.6.2
##  [55] gplots_3.0.1.1           RColorBrewer_1.1-2       ggridges_0.5.1
##  [58] TFisher_0.2.0            Rcpp_1.0.3               plyr_1.8.5
##  [61] zlibbioc_1.32.0          purrr_0.3.3              RCurl_1.95-4.12
##  [64] pbapply_1.4-2            viridis_0.5.1            cowplot_1.0.0
##  [67] zoo_1.8-6                ggrepel_0.8.1            cluster_2.1.0
##  [70] magrittr_1.5             data.table_1.12.8        lmtest_0.9-37
##  [73] RANN_2.6.1               mvtnorm_1.0-11           fitdistrplus_1.0-14
##  [76] lsei_1.2-0               mime_0.8                 evaluate_0.14
##  [79] xtable_1.8-4             compiler_3.6.2           tibble_2.1.3
##  [82] KernSmooth_2.23-16       crayon_1.3.4             R.oo_1.23.0
##  [85] htmltools_0.4.0          pcaPP_1.9-73             later_1.0.0
##  [88] tidyr_1.0.0              rrcov_1.4-9              RcppParallel_4.4.4
```

```
##  [91] WriteXLS_5.0.0          MASS_7.3-51.5           rappdirs_0.3.1
##  [94] Matrix_1.2-18           R.methodsS3_1.7.1       gdata_2.18.0
##  [97] metap_1.2               igraph_1.2.4.2          pkgconfig_2.0.3
## [100] sn_1.5-4                registry_0.5-1          numDeriv_2016.8-1.1
## [103] plotly_4.9.1            vipor_0.4.5             rngtools_1.4
## [106] pkgmaker_0.27           multtest_2.42.0         XVector_0.26.0
## [109] bibtex_0.4.2.1          doRNG_1.7.1             stringr_1.4.0
## [112] digest_0.6.23           sctransform_0.2.1       RcppAnnoy_0.0.14
## [115] tsne_0.1-3              rmarkdown_2.0           leiden_0.3.1
## [118] uwot_0.1.5              DelayedMatrixStats_1.8.0 shiny_1.4.0
## [121] gtools_3.8.1            lifecycle_0.1.0         nlme_3.1-143
## [124] jsonlite_1.6            BiocNeighbors_1.4.1     viridisLite_0.3.0
## [127] pillar_1.4.3            lattice_0.20-38         fastmap_1.0.1
## [130] httr_1.4.1              plotrix_3.7-7           DEoptimR_1.0-8
## [133] survival_3.1-8          glue_1.3.1              png_0.1-7
## [136] class_7.3-15            stringi_1.4.3           BiocSingular_1.2.1
## [139] caTools_1.17.1.3        irlba_2.3.3             e1071_1.7-3
## [142] future.apply_1.3.0      ape_5.3
```

References

"A Beginner's Guide to Single-Cell Rna Sequencing." n.d. https://bitesizebio.com/40996/beginners-guide-to-single-cell-rna-sequencing/.

Auguie, Baptiste. 2017. *GridExtra: Miscellaneous Functions for "Grid" Graphics*. https://CRAN.R-project.org/package=gridExtra.

Duò, Angelo, Mark D. Robinson, and Charlotte Soneson. 2018. "A Systematic Performance Evaluation of Clustering Methods for Single-Cell Rna-Seq Data." *F1000Research*. F1000 Research Limited. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6134335/.

Kiselev, Vladimir Y. 2019. *SC3 Package Manual*. https://bioconductor.org/packages/release/bioc/vignettes/SC3/inst/doc/SC3.html.

Kiselev, Vladimir Y., Kristina Kirschner, Michael T. Schaub, Tallulah Andrews, Andrew Yiu, Tamir Chandra, Kedar N. Natarajan, et al. 2017. "SC3: Consensus Clustering of Single-Cell Rna-Seq Data." *Nature Methods*. Nature Research. https://www.nature.com/articles/nmeth.4236.

McCarthy, Davis J., Kieran R. Campbell, Aaron T. L. Lun, and Quin F. Willis. 2017. "Scater: Pre-Processing, Quality Control, Normalisation and Visualisation of Single-Cell RNA-Seq Data in R." *Bioinformatics* 33 (8): 1179–86. https://doi.org/10.1093/bioinformatics/btw777.

Microsoft, and Steve Weston. 2019. *Foreach: Provides Foreach Looping Construct*. https://CRAN.R-project.org/package=foreach.

Satija Lab. 2019. *Seurat - Guided Clustering Tutorial*. https://satijalab.org/seurat/v3.1/pbmc3k_tutorial.html.

Scrucca, Luca, Michael Fop, Thomas Brendan Murphy, and Adrian E. Raftery. 2016. "mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models." *The R Journal* 8 (1): 205–33. https://journal.r-project.org/archive/2016-1/scrucca-fop-murphy-etal.pdf.

Stuart, Tim, Andrew Butler, Paul Hoffman, Christoph Hafemeister, Efthymia Papalexi, William M Mauck III, Marlon Stoeckius, Peter Smibert, and Rahul Satija. 2018. "Comprehensive Integration of Single Cell Data." *bioRxiv*. https://doi.org/10.1101/460147.

Wickham, Hadley, Romain François, Lionel Henry, and Kirill Müller. 2019. *Dplyr: A Grammar of Data Manipulation*. https://CRAN.R-project.org/package=dplyr.

Zappia, Luke, Belinda Phipson, and Alicia Oshlack. 2017. "Splatter: Simulation of Single-Cell Rna Sequencing Data." *Genome Biology*. https://doi.org/10.1186/s13059-017-1305-0.